Distributed Systems

Summary

Authors

Katharina Tschirky

Date

15 June 2026

Table of Contents

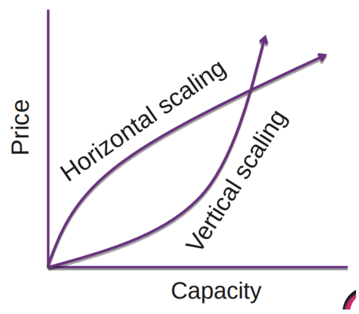
1. Week 1 - Scaling

1.1. Distributed Systems Motivation - Scaling

- Vertical (scale up), more memory, faster CPU
  - Single Core
    - Amdahl's Law: Your total speedup is limited by the part of the work that cannot be parallelized
    - S = 1 / ((1 - P) + P/N)
    - S = Speedup, P = parallelizable fraction (0–1), N = number of parallel units
    - Example: P = 95% -> even with infinite processors, the speedup can never be more than 20x
    - Thus the sequential part is always the bottleneck e.g in real word DB locks limit throughput regardless of server count
- Horizontal (scale out), more machines

Machine Learning:
- Current trend: scale horizontally
- ML with vertical scaling is not (yet) feasible

Economics:
- Initially scaling vertically is cheaper until HW is maxed out
- Currently: RAM prices increasing, fast servers



Horizontal Scaling
+ Lower cost with massive scale
+ Easier to add fault-tolerance
+ Higher availability
- Adaption of software required
- More complex system, more components involved

Vertical Scaling
+ Lower cost with small scale
+ No adaption of software required
+ Less complexity
- HW limits for scaling
- Risk of HW failure causing outage
- More difficult to add fault-tolerance

1.2. Vertical Scaling Performance

Notable laws:
- Moore's Law: nr. of transistors doubles

every 2 years
- Nielsen's Law: a high-end user's

connection speed grows by 50% per year
- Kryder'S Law: Disk density doubles every 13 months
- Best principal for small and simple applications -> Simple website with a few DB calls is not HW intensive
- Bandwidth usually grows slower than computer power (conservative telecom/reluctant users).
- Wirth's law (1995): Hardware gets faster but software complexity and resource demands grow even faster

1.3. Energy: The new scaling constraint

- AI-specific hardware will surpass bitcoin scaling-economics
- ChatGPT query: 0.3W

- 100 queries roughly 15min of netflix
- Efficiency is improving
- Time savings give indirect energy savings
- Rebound effect -> users generate more and more queries

2. Week 2 - Location, Latency and Redundancy

2.1. Distributed Systems Motivation - Location

- Hardware might get faster but latency stays
- Nothing is faster than the speed of light -> you will always have some latency
- Reduce latency by placing systems closer to users (or by using a CDN)
- Legal requirements for data location (GDPR)

Latency: time for signal to travel from source to destination and back (round-trip time)

Traceroute: Tool used to visualize hops
- Ping only shows total RTT
- Traceroute shows the path
- Sends package with increasing TTL (Each router makes TTL - 1 when TTL = 0 then ICMP "Time Exceeded" is sent)

2.1.1. Speed of Light

- There is a difference between practical and theoretical limit (300ms vs 110ms)
- Usually no direct paths from one location to another (e.g there is no straight cable from my home to my friends home in amsterdam)
- Signals can only travel at the speed of light inside a vacuum

2.1.1.1. Different fiber types

Single mode fibers: Provide lower latency, one main path, less delay spread

Multimode fibers: Many paths, some longer, more delay spread

The refractive index of glass depends slightly on the wavelength of the light. This is called chromatic dispersion. Different wavelengths travel at slightly different speeds, so the latency depends on what optical wavelength is used

Hollow core fiber: Has less latency, guides light through air and not through solid glass. Thus, light can travel closer to its vacuum speed.

Copper: Propagates faster but not much (Good copper cable > standard fiber)

Quick facts:
- Network latency has a physical lower limit because signals cannot travel faster than light.
- Fiber is slower than space/air because light travels through glass at about 200,000 km/s, not 300,000 km/s.
- Real network latency is higher than the theoretical minimum due to routing, queuing, protocol overhead, traffic inspection and signal repeating.
- Satellites can sometimes be faster than fiber because they can take a more direct route and signals travel faster through air/space. However, weather conditions can affect signal strength
- Wi-Fi is not the lowest-latency option because it adds waiting time, acknowledgements, retransmissions and MAC-layer processing.
- Main idea: latency depends on distance, signal speed, routing path and protocol overhead.

2.2. Motivation for Distributed Systems - Redundancy / Fault-tolerance

- Any hardware will crash eventually -> it's not a question of if but when
- Random bit flips in memory

    - Might happen for DNS names in your memory, that's why many providers have **Bitsquat Domains** (e.g ikamai.net -> akamai.net)
- Cosmic rays may be blamed for an electronic voting error (bit flip)
- Techniques like replication, checksums, redundancy, consensus and monitoring help keep systems reliable.
- HDD break, SSDs wear out (use NAND cells with limited lifetime, they have spare NAND)

-Distributed Systems provide a solution
- Multiple machines provide redundancy
- When one machine fails, others can take overhead
- Load can be redistributed among remaining machines
- System continues to function despite individual failures

Trade-off consideration:
- Distributed systems add complexity
- Use only when benefits outweigh the added complexity
- Redundancy and fault tolerance must justify the complexity

One possible solution:
- Error-correcting code memory (ECC)
- Hamming Code, correct 1 bitflip / detect 2 bitflips
- Used mainly for servers
- Consumer: DDR5 has on-die ECC but weaker than traditional ECC (corrects only inside the chip at rest but not on the data bus in transit)

NAND flash memory types:
- SLC = Single-Level Cell -> Stores 1 bit per cell.
- MLC = Multi-Level Cell -> Usually stores 2 bits per cell.
- TLC = Triple-Level Cell -> Stores 3 bits per cell.
- QLC = Quad-Level Cell -> Stores 4 bits per cell.

The tradeoff is: more bits per cell means cheaper and denser storage, but lower endurance and usually slower writes. Caching with SLC → files / cells that are frequently changed, store on SLC, once they don't change that often move to MLC/TLC/QLC

The goal is to level the wear and distribute write and erase operations across all memory cells.

2.3. Fault Tolerance

Seacables provide 99% of data connectivity. Network outages do happen from time to time.

3. Key takeaways (Week 1-2) - Why do we need Distributed Systems

3.1. Why do we need distributed systems?

We need distributed systems mainly for scaling, location, and fault-tolerance. Distributed systems add complexity, so this should be avoided unless needed.

3.1.1. Scaling (if one machine is not enough)

Scaling means increasing system capacity.

Vertical scaling means using a stronger machine, for example more memory or a faster CPU.

Advantages:
- Lower cost at small scale
- No software adaptation required
- Less complexity

Disadvantages:
- Hardware limits
- Hardware failure can cause an outage
- Fault-tolerance is harder
- Speedup is limited by sequential work, see Amdahl's Law

Horizontal scaling means using more machines.

Advantages:
- Lower cost at massive scale
- Easier to add fault-tolerance
- Higher availability

Disadvantages:
- Software adaptation required
- More complex system with more components

3.1.2. Location (to move closer to the user)

Location matters because hardware can get faster, but latency remains limited by the speed of light. Distributed systems can reduce latency by placing systems closer to users, for example with a CDN.

Other reason for location:
- Legal requirements for data location, such as GDPR

Latency is the time for a signal to travel from source to destination and back.

Important points:
- Real network paths are usually not direct
- Fiber is slower than vacuum because light travels through glass
- Single-mode fiber has lower latency than multimode fiber
- Hollow-core fiber can have lower latency because light travels mostly through air
- Copper can be slightly faster than standard fiber, but not by much
- Traceroute shows network hops, while ping only shows total RTT

3.1.3. Fault-tolerance (HW will fail eventually)

Fault-tolerance means preparing for failures. Hardware will eventually fail, and at scale rare failures become expected.

Examples:
- Random bit flips in memory
- HDDs break
- SSDs wear out
- Network outages happen, including sea cable failures

Distributed systems help because:
- Multiple machines provide redundancy
- If one machine fails, others can take over
- Load can be redistributed
- The system can continue despite individual failures

Reliability techniques from Week 2:
- Replication
- Checksums
- Redundancy
- Consensus
- Monitoring
- ECC memory

4. Week 3

4.1. Key takeaways

4.1.1. What is the difference of VM / Container?

A VM runs like a real computer with its own guest OS on a hypervisor. A container is an isolated user-space instance that shares the host OS/kernel.

4.1.2. How does docker work (container implementation)?

Docker packages software into containers. Containers are isolated with Linux mechanisms like network namespaces, cgroups, OverlayFS, seccomp profiles, and Linux capabilities.

4.1.2.1. Best practices

Keep images small and up to date, scan for vulnerabilities, do not expose the Docker daemon socket, do not run as root, limit capabilities/resources, and use read-only filesystems/volumes where possible.

4.1.3. What is docker-compose, and how to run multiple services

Docker Compose runs multiple containers together, for example services, clients, databases, or load balancers.

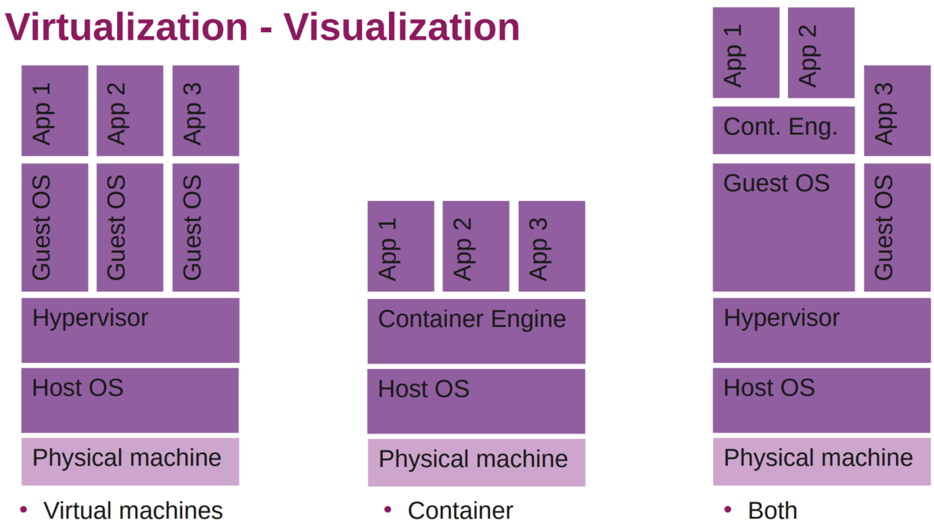
Define the services in a `docker-compose.yml`, then Docker Compose starts and connects them as one application setup.

4.2. Containers and VMs

4.2.1. Virtualization

Creation of a virtual machine that acts like a real computer with an operating system.

- **Host Machine:** machine where the virtualization software runs
- **Guest machine:** Virtual machine
- **Hypervisor:** Runs virtual machine
  - Type 1: bare metal e.g VMware ESXi
  - Type 2: hosted e.g VirtualBox
- Run a modified OS with Intel VT-x and AMD-V, or paravirtualized if not present (VM should not access memory directly)
- Needs to be the same architecture (otherwise needs to be emulated)
- Virtual Desktop Infrastructure (VDI)
- Containers
  - Isolated user-space instances
  - OS support: isolations



4.2.2. VM vs Container

+ Reduced size of snapshots 2MB vs 67MB
+ Quicker spinning up apps
+/- Available memory is shared
+/- Process-based isolation (share same kernel)

Use Case: complex application setup, with container less complex configuration Providers: ECS, Google Cloud Run, Digital Ocean App Platform

+ App can access all OS resources
+ Live migrations
+/- Pre allocates memory
+/- Full isolation

Use Case: better hardware utilization / resource sharing Providers: EC2, Virtual Machines, Computer Engine, Droplets

4.2.3. Prices

Cloud VM pricing has several models:
- On-demand: Pay as you go for the VM plus related costs like data transfer and IP addresses.
- Spot instances: Cheaper, discounted compute capacity when providers have unused resources, but availability is less reliable.
- Reserved instances: Lower prices in exchange for committing to usage over a longer period.

Main takeaway: VM cost optimization requires regularly comparing providers and pricing models, since costs and resource bundles differ and can change over time.

4.2.4. Firecracker vs VirtualBox

Firecracker is a lightweight MicroVM with minimal devices, no BIOS/UEFI boot, very fast startup and low memory overhead.

VirtualBox is a full type 2 VM with broad device emulation, a normal BIOS/UEFI boot chain, slower startup and pre-allocated memory.

Main takeaway: Firecracker is optimized for speed and efficiency, VirtualBox is optimized for full VM functionality.

4.3. Docker

4.3.1. Container Introduction

- **LXC (Linux Containers):** Lower abstraction level and direct use of Linux kernel features
- **systemd-nspawn:** Part of the systemd project, minimalist container manager

- **Solaris Zones:** Oracle/Sun-specific container technology
- **Linux-VServer:** Kernel patch for Linux, older virtualization technology
- **OpenVz:** Operating system-level virtualization, popular hosting tool
- **Singularity:** Scientifically oriented container solution, HPC-friendly
- **Podman:** Docker alternative / replacement
- **Docker:** / docker-compose for software development

4.3.2. Docker Introduction

- Docker is a containerization platform that packages software into containers (existing images can be found on docker hub)
- Containers are isolated from each other with linux-based isolation mechanisms, they only communicate over well-defined channels
  - Network namespaces
    - Each container gets its own network environment (e.g IP, port, interfaces)
  - Cgroups
    - Control how many resources a container can use (e.g CPU, RAM, disk I/O)
  - OverlayFS
    - Lets docker build images in layers
    - Container can share read-only image layers and add a small writable layer on top
  - Security
    - seccomp profiles
      - Restrict which linux system calls a container is allowed to use
    - Linux capabilities
      - Split root privileges into smaller permissions
- Docker can be used on macOS or Windows
  - WSL2 runs a lightweight linux VM, has better isolation than bare metal linux but I/O is slower

4.3.3. Docker Examples

- If no image version is specified, Docker uses the `latest` tag.
- Docker Hub is the main repository for container images.
- GUIs like Docker Desktop or Podman Desktop can also manage Docker.

- Install Docker and test it:
  - `docker run hello-world`
- List installed images:
  - `docker images`
  - `docker images -a`
- Remove an image:
  - `docker rmi hello-world`
  - `docker rmi <image-id>`
- Save and inspect an image:
  - `docker save hello-world -o test.tar`
  - `tar xf test.tar`
  - `tar xf <layer-id>/layer.tar`
- Run extracted program:

  - `./hello`
- List containers:
  - `docker ps -a`
- Remove a container:
  - `docker rm <container-id>`

4.3.4. Details

- Docker images use filesystem layers (FS Virtualization).
- OverlayFS combines multiple layers into one visible filesystem.
- Example Dockerfile:

```
FROM alpine
ADD hello.sh .
ENTRYPOINT ["sh", "hello.sh"]
```

- Example script (hello.sh):

```
#!/bin/sh
echo "Hallo"
```

- Meaning:
  - `FROM alpine`: use Alpine Linux as the base image.
  - `ADD hello.sh .`: copy `hello.sh` into the image.
  - `ENTRYPOINT ["sh", "hello.sh"]`: run the script when the container starts.
- Example commands:
  - `docker build . -t test`
  - `docker run test`
  - `docker save test:latest > test.tar`
- The image has at least two layers:
  - Alpine base layer, including BusyBox, musl libc, crypto/ssl, etc.
  - New layer containing `hello.sh`.
- If the input does not change, Docker reuses the cached layer.

4.3.5. OverlayFS

OverlayFS is a Linux filesystem that shows multiple directories as one combined directory.

It has three main parts:

- **lowerdir:** the base layer. Usually read-only. In Docker, this is like the image layer.
- **upperdir:** the writable layer. New or changed files are stored here.
- **workdir:** a temporary working directory OverlayFS uses internally when moving or updating files.
- **overlay mount:** the final merged view that the user/container sees.

Example:

```
cd /tmp
mkdir lower upper workdir overlay

sudo mount -t overlay -o \
lowerdir=/tmp/lower,\
upperdir=/tmp/upper,\
```

```
workdir=/tmp/workdir \
none /tmp/overlay
```

`/tmp/overlay` becomes the mount point and shows the combined contents of lower and upper

If a file exists in lowerdir and you edit it, OverlayFS does not modify the read-only lower layer. Instead, it copies the file into upperdir and changes it there.

If you delete a file that exists only in lowerdir, OverlayFS cannot really delete it from the read-only layer. So it creates a marker in upperdir saying "this file is deleted." The file then disappears from the merged overlay view.

You can also have multiple lower layers:
`lowerdir=/tmp/lower1:/tmp/lower2 /tmp/overlay`

Main idea: OverlayFS lets Docker keep image layers read-only while storing container changes separately in a writable upper layer.

4.3.6. Cgroups

Control Groups: limits, isolates, prioritization of CPU, memory, disk I/O, network

Docker uses cgroups internally for container resource control.

- Example: create two CPU groups:

  ```
  cgcreate -g cpu:red
  cgcreate -g cpu:blue
  ```

- Assign different CPU weights:

  ```
  echo -n "20" > /sys/fs/cgroup/blue/cpu.weight
  echo -n "80" > /sys/fs/cgroup/red/cpu.weight
  ```

- Run shells inside the groups:

  ```
  cgexec -g cpu:blue bash
  cgexec -g cpu:red bash
  ```

- Test CPU competition on one core:

  ```
  taskset -c 0 sha256sum /dev/urandom
  ```

- Docker example:

  ```
  docker run --name=low_prio --cpuset-cpus=0 --cpu-shares=20 alpine sha256sum /dev/
  urandom
  docker run --name=high_prio --cpuset-cpus=0 --cpu-shares=80 alpine sha256sum /dev/
  urandom
  ```

Main idea: cgroups let Linux and Docker isolate, limit, and prioritize resources between processes or containers.

4.3.7. Separate networks

Linux network namespaces provide isolation of the system resources associated with networking.

Create a namespace:

```
ip netns add testnet
ip netns list
```

Create a virtual Ethernet pair between host and namespace:

```
ip link add veth0 type veth peer name veth1 netns testnet
ip link list
```

Run commands inside the namespace:

```
ip netns exec testnet <cmd>
```

Configure IP addresses and enable interfaces:

```
ip addr add 10.1.1.1/24 dev veth0
ip netns exec testnet ip addr add 10.1.1.2/24 dev veth1
ip link set dev veth0 up
ip netns exec testnet ip link set dev veth1 up
```

Run a server inside the namespace:

```
ip netns exec testnet nc -l -p 8000
```

Connect the namespace to the outside network:

```
echo 1 > /proc/sys/net/ipv4/ip_forward
ip netns exec testnet ip route add default via 10.1.1.1 dev veth1
iptables -t nat -A POSTROUTING -s 10.1.1.0/24 -o enp6s0 -j MASQUERADE
iptables -A FORWARD -j ACCEPT
```

Main idea: network namespaces give containers separate network environments while still allowing controlled communication with the host or internet.

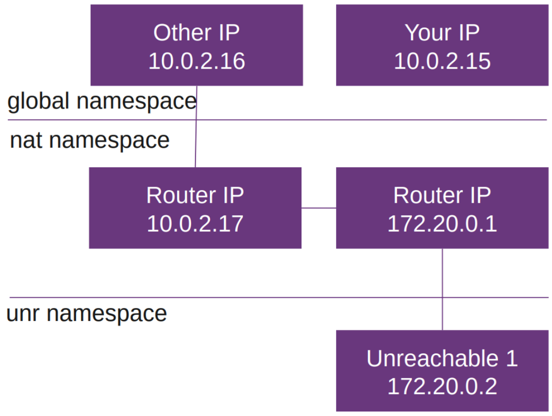
4.3.8. Connectivity, Security and Robustness

- **NAT (Network Address Translation):** Multiple devices share one public IP -> devices are not directly reachable
- **Hole punching:** Two peers behind NAT send coordinated packets to create NAT mappings, enabling a direct connection
  - P2P / Hole Punching development (in the old days)
  - Currently: network namespaces

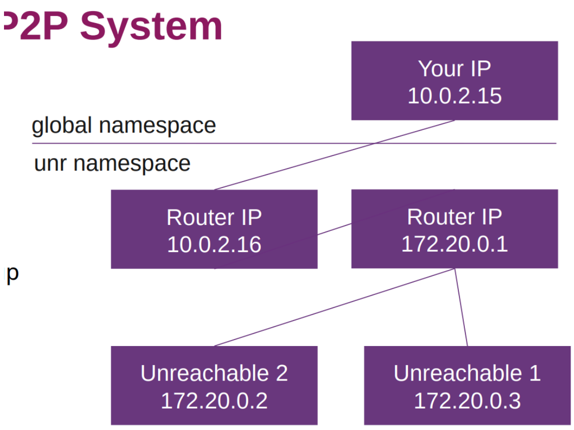
4.3.8.1. P2P System

- **veth**: Virtual Ethernet Device, like a virtual network cable between namespaces.
- Used to build a local P2P testbed with peers that are reachable or hidden behind NAT.
- Example setup: global namespace `10.0.2.15`, router/NAT `10.0.2.16` and `172.20.0.1`, unreachable peers `172.20.0.2` and `172.20.0.3`.
- Create namespace, veth pair, and router interface:
  - `unr` is the isolated network namespace for the unreachable peer side.
  - `nat_lan` and `nat_wan` are two ends of one virtual Ethernet cable.
  - `nat_lan` is moved into `unr`, while `nat_wan` stays in the global namespace.

```
ip netns add unr
ip netns list

ip link add nat_lan type veth peer name
nat_wan
ip link set nat_lan netns unr

ip address add 10.0.2.16/24 dev nat_wan
ip link set nat_wan up

ifconfig
ping <ip>

ip netns exec unr ip address add
172.20.0.1/24 dev nat_lan
ip netns exec unr ip link set nat_lan up
```



- Add unreachable peer interface and routes:
- `unr1` is a dummy interface that represents an unreachable peer inside the namespace.
- `172.20.0.2` is the peer IP, while `172.20.0.1` is the router/gateway inside `unr`.
- The loopback interface `lo` is enabled because many programs expect localhost to exist.
- The route commands tell traffic how to leave the namespace and how the host can reach the namespace network.




```
ip netns exec unr ip link add unr1 type
dummy
ip netns exec unr ip address add
172.20.0.2/24 dev unr1
ip netns exec unr ip link set unr1 up

ip netns exec unr ifconfig

ip netns exec unr ip link set lo up
ip netns exec unr route add default gw
172.20.0.1

ip route add 172.20.0.1 dev nat_wan
```

Main idea: network namespaces and veth pairs can simulate real P2P/NAT situations on one machine.

4.3.9. Docker Compose

Dockerfiles build individual container images. Docker Compose runs multiple containers together as one application setup.

- Dockerfile:
  - Used to build your own image.
  - Keep images small by only copying required files.
  - Docker caches layers aggressively, so unchanged steps are reused.

    - Squash images: `COPY --from=initial / /`

- Example multi-stage Dockerfile:

```
FROM golang:alpine AS builder
WORKDIR /build
COPY server.go .
RUN go build server.go

FROM alpine
WORKDIR /app
COPY --from=builder /build/server .
ENTRYPOINT ["./server"]
```

- Meaning:
    - First stage `builder`: contains Go and builds the binary.
    - Second stage `alpine`: small final runtime image.
    - `COPY --from=builder`: copies only the compiled binary into the final image.

- Docker Compose:
    - Used to deploy multiple containers, e.g. server, client, database, load balancer.
    - Configures services in one `docker-compose.yml`.
    - Provides lightweight orchestration, e.g. starting services that depend on others.

- Example `docker-compose.yml`:

```
services:
  server1:
    build: .
  client:
    image: alpine
    entrypoint: >
      sh -c "sleep 3 && echo hallo | nc server1 8081"
```

- Meaning:
    - `server1` is built from the local Dockerfile.
    - `client` uses the Alpine image.
    - The client waits briefly, then sends `hallo` to `server1` on port `8081`.

Main idea: Dockerfiles define images. Docker Compose defines how multiple containers are started and connected.

4.3.10. Docker Security

Best Practices:
- Keep images small, up to date / attack surface
    - **alpine / distroless:** Alpine is preferred because it creates very small, minimal Docker images that are faster to transfer, have fewer unnecessary packages and usually a smaller security surface.
- Check your image for vulnerabilities, snyk, trivy
- Do not expose the Docker daemon socket (even to containers)
- Set a user - do not run as root
    - Needs more configuration but good advice
- Limit capabilities (grant only what is needed by a container) -> seems overkill
- Limit resources (memory, CPU, file descriptors, processes, restarts)

  - Good for production -> docker does not have a hard memory limit. Docker kills process that goes over limit, no pressure for GC if not docker aware
- Set file system and volumes to read-only

4.3.11. How to debug

What is your base image:
1. Ubuntu, Debian, Alpine, distroless etc.
2. GNU libc (e.g distroless) vs. musl (e.g alpine)
   - Glibc (GNU C library): wide adaptation, many distros use it, larger binary
   - Musl (small C standard library): less used, smaller binary
   - If glibc is needed in alpine then a compatible busybox docker image or installing gcompat can help
   - Building outside docker container might fail because of missing libraries etc.

Alpine Tools:
- Busybox: nc, ping, sha256sum, wget, netstat
- Reach other container: ping containername
- Service bound to localhost? Cannot run outside docker, use 0.0.0.0

Other stuff:
- docker system prune -a (attention)
  - Deletes unused docker data to free disk space (stopped containers, unused networks, unused images, build cache)
- Check logfiles
- Performance: multi-stage, .dockerignore
- Docker: aggressive caching - wget is cached, use version numbers

5. Week 4

5.1. Key takeaway

5.1.1. What is a monorepo, what is a polyrepo?

Monorepo: One repository contains multiple parts of a project, e.g. frontend and backend in separate folders.

Polyrepo: A project is split across multiple repositories, e.g. one repo for frontend and one for backend.

5.1.2. When to use which type?

When to use monorepo: Good when projects are tightly connected, need shared code, or often change together.

When to use polyrepo: Good when projects are independent, need separate access control, or are managed by different teams.

5.1.3. How do AI coding agents change the monorepo vs. polyrepo decision?

Monorepos are easier for agents because they can see all related code at once. Polyrepos need extra setup and more context management.

5.2. Project setup

General Rules:

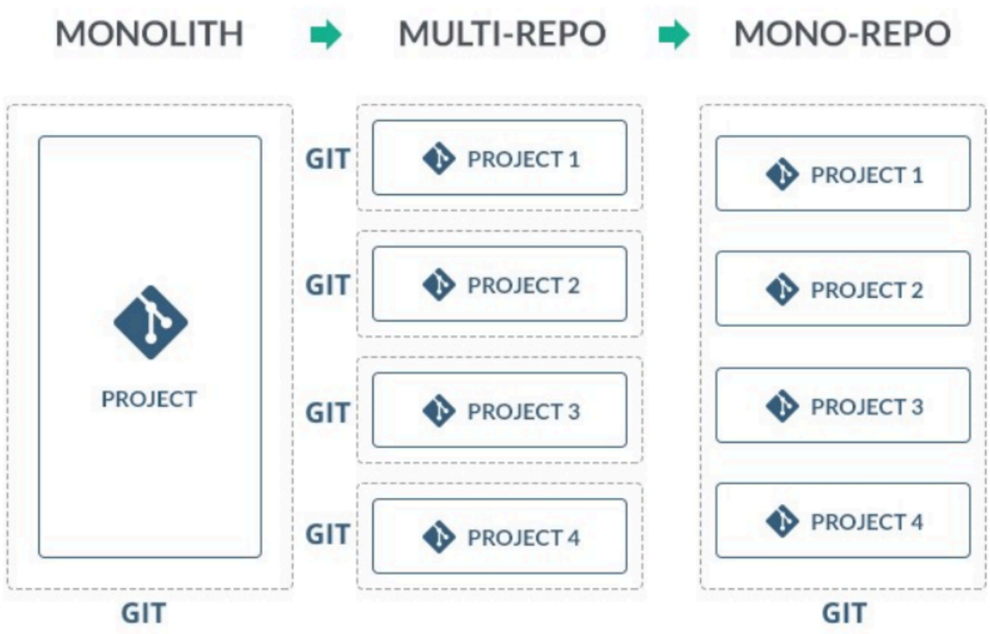
1. Be consistent
2. Other projects always prefer other structures
3. A perfect structure does not exist
4. Every project has a readme
5. Follow best practices (e.g java in src/main/java)

Split up:

- Backend, frontend in separate repositories
- 1 technology per split for simple projects
  - Most likely you won't have a frontend mix of frontend technologies e.g, Angular with Vue
  - Sometimes you have a script directory with different languages (bash, javascript)
- Keep config out of code (secrets, API keys -> .env not committed)
- More complex setups -> multiple backends, multiple packages

5.3. Monorepo (OneRepo / UniRepo)

- One repository for all projects
  - 1 sub-directory with frontend, 1 sub-directory with backend etc.
- Easier setup for AI agents



5.4. Polyrepo (Manyrepo / Multirepo)

- Multiple repository for a project
- Frontend in different repo than backend
- Sync via git submodules or via bash script
  - Submodules: can also be used as dependency management
- Sync with repo or via bash script

5.5. Pro/Cons

Monorepo:

- Tight coupling of projects
  - E.g., generating openapi.yml from backend, generate types for frontend → simply copy, or tRPC
- Everyone sees all code / commits
- Encourages code sharing within organization
- AI agents: full cross-project context (backend + frontend in one PR)
- Atomic commits for cross-project changes
- Scaling: large repos, specialized tooling
- Risk: cross-package dependencies (one change can affect many service -)

Polyrepo:

- Loose coupling of projects
  - If you want to generate openapi.yml, you need access from the backend repository to the frontend (e.g., curl+token)
- Fine grained access control
- Encourages code sharing across organizations
- AI agents: possible (–add-dir), but extra setup and token cost
- Synthetic Monorepo (Nx): bridges repo boundaries without moving code
- Scaling: many projects, special coordination

5.6. Tools

- Monorepos need tools to manage builds, tests, and dependencies across many packages.
- Tools include pnpm workspaces, Turbo, Nx, Gradle, Bazel, Buck2, or simpler scripts like make/just.
- A key feature is caching: if inputs did not change, the command does not need to run again.
- Main goal: speed up builds and reduce repeated work.

6. Week 5 - Load Balancing

6.1. Key takeaways

6.1.1. What types of LB exists?

Hardware, software, and cloud-based load balancers. Software load balancing can happen at L2/L3, L4, L7, DNS level, or with Anycast.

6.1.2. Which one to pick?

It depends on control and use case: hardware load balancers are mainly useful if you control your datacenter; cloud load balancers are pay-per-use with many offerings; software load balancers give options like Traefik, Caddy, Nginx, HAProxy, MetalLB, and DNS/Anycast approaches.

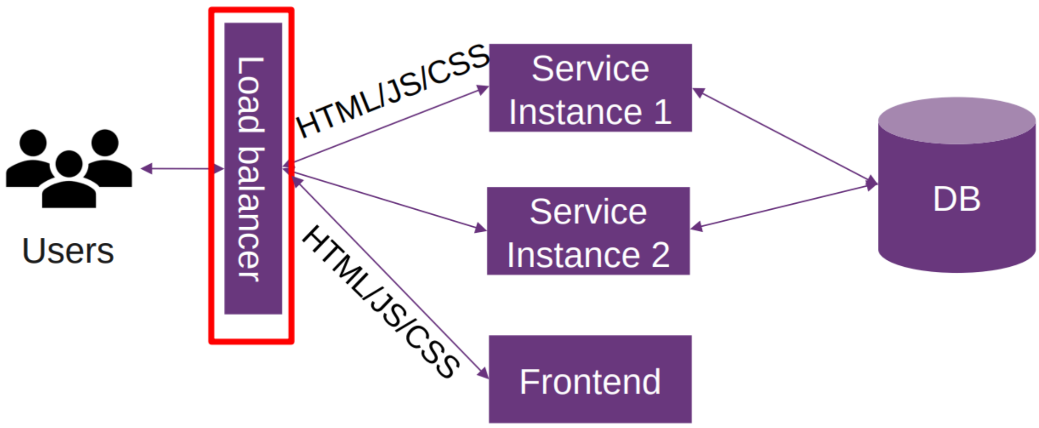
6.1.3. What is currently “state-of-the-art”?

Modern software load balancers/reverse proxies such as Traefik, Caddy, Nginx, and HAProxy, plus cloud-based L4/L7 load balancers. Additionally, you should measure performance yourself with realistic, reproducible benchmarks.

6.1.4. CORS

CORS means Cross-Origin Resource Sharing. Browsers restrict cross-origin HTTP requests from scripts. A common solution is using a reverse proxy so frontend assets and API appear under the same origin. Another workaround is setting Access-Control-Origin during development.

6.2. Load Balancing



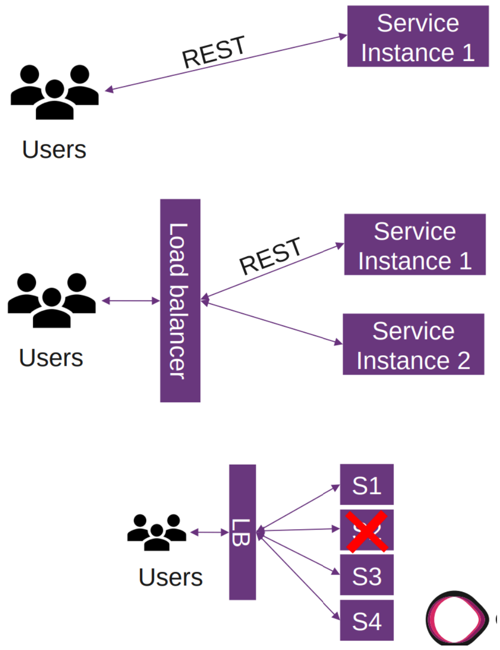
What is load balancing?

- Distribution of workload across multiple computing resources
  - Workloads (requests)
  - Computing resources (machines)
- Distributes client requests or network load efficiently across multiple servers
  - E.g service gets popular, high load on service

-> Horizontal scaling

Why load balancing?

- Ensures high availability and reliability by sending requests only to servers that are online
- Provides the flexibility to add or subtract servers as demand dictates



6.2.1. Types of load balancer (Cloud, Hardware, Software)

6.2.1.1. Hardware load balancer

- HW-LB use proprietary software, which often uses specialized processors
  - Less generic, more Performance
  - Some use open-source SW, e.g HA-proxy
- E.g loadbalancer.org, cisco, kemp
- Only if you control your datacenter

6.2.1.2. Software load balancer

- L2/L3: MetalLB (Kubernetes bare-metal, standard solution for type: LoadBalancer on bare-metal/kind clusters without a cloud LB), Seesaw (not widely used)
- L4: HAProxy (desc), Nginx, Gobetween, Traefik
- L7: Enovy (C++), HAProxy (C), Nginx (C), caddy (golang), Gobetween (golang), Eureka (java) - services register at eureka, traefik (golang)

Measure performance yourself with realistic, reproducible benchmarks instead of relying only on opinions or generic benchmark results.

L4 vs L7 Load Balancing:

- L7: more resource intensive, can make smarter decisions
- L7: terminates TLS and inspects HTTP (encryption overhead)

DNS Load Balancing (L7):

- Round-robin DNS
  - Easy to setup, client-side selection
  - Drawback: negative caching impact
  - Used in Bitcoin Core: dig dnsseed.emzy.de
  - Example:

```
www A 188.40.119.115
lb  A 188.40.119.115
lb  A 152.96.80.48
```

- Split horizon DNS
  - Different DNS information, depending on source of DNS request
- Layer 3: Anycast
  - Multiple servers announce the same IP address from different location
  - Requires ownership of an AS and BGP routing

Main idea: software load balancing can happen at HTTP level, DNS level, or IP routing level, each with different complexity, performance, and control.

6.2.1.3. Cloud-based load balancer

- Pay for use, many offerings
- AWS
  - Application Load Balancer ALB (L7)
  - Network Load Balancer (L4)
  - Classic Load Balancer (legacy)
- Google Cloud (L3, L4, L7)
- Cloudflare (L4, L7)
- DigitalOcean (L4)
- Azure (L4, L7)

6.2.2. Load Balancing Algorithms

- Easiest: round-robin / random
  - distributing requests evenly by rotating through a list of servers.
  - Make sure your services are stateless
- Stateless: don't store anything in the service
  - If you do, you need a stick session (avoid this) - same user to same service
  - E.g cookie, ip_hash - send to same machine
- Health checks: tell your load balancer if you are running low on resources
  - Active: send active probes e.g every 3s
  - OOB - out of band (API to check health) e.g necessary with DB, as connection may be OK, but database not
  - Passive: only check with request
- Different behaviour:
  - Nginx: passive, caches requests, so if an upstream fails, it uses another
  - Caddy: passive, does not cache, but marks upstream as failed for the next request

Round-robin:
- Loop sequentially
- Simple algorithm, often default
- May drop requests on congested nodes

Weighted round-robin:
- You can put weighted in from everything
- Some servers are more powerful -> more powerful machines get more work
- But high variance in server load may drop requests

Least connections / fewest current connections to clients:
- Load balancer sends the request to the server that currently has the fewest outstanding requests
- Keep track of outstanding requests
- Not best for latency -> does not know if requests in queue are fast or slow

Peak exponentially weighted moving average:
- Measures average on how long a server takes to respond and gives importance to recent measurements
- Considers latency
- Increased complexity

6.2.3. Traefik
- Open Source, software-based load balancer
  - L4/L7 load balancer
  - Traefik is written in Go
  - Can support authentication mechanisms, especially for the dashboard
  - HTTP/3 support
- Has a built-in dashboard
- Traefik provides an official docker image

6.2.4. Service
- As a start -> is stateful
- Written in Go
- Stickiness with cookies
- Weighted round round-robin
  - Load balances between services and not between servers

6.2.5. Caddy
- Configuration: dynamic
  - Static: Caddyfile
- One liners:
  - Quick local file server: caddy file-server
  - Reverse proxy: caddy reverse-proxy –from example.com –to localhost:9000
- Open source, software-based load balancer
- L7 load balancer
- Reverse proxy
- Static file server
- HTTP/1.1, HTTP/2, HTTP/3
- Caddy is on docker hub
- Automatic HTTPS (Let’s Encrypt)

6.2.6. Nginx
- Free and commercial version
- Fast webserver, 35% market share
- HTTP proxy, mail proxy, reverse proxy, load balancer
- No active health checks, no sticky sessions

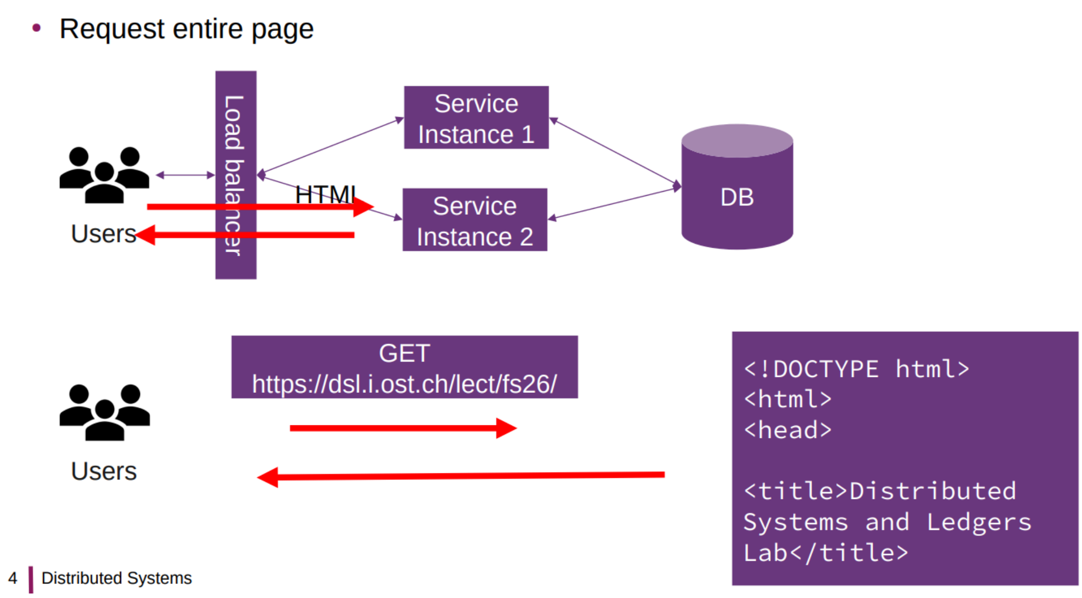
6.2.7. HAProxy
- L4 and L7 load balancer and reverse proxy
  - Open source option: commercial support

- Fast performance
- Configure and run: /etc/init.d/haproxy start
  - Algorithms: roundrobin, leastconn, source
  - Sticky session: cookie
  - check -> health checks (passive / active)
- Primary/secondary: backup server only receives traffic when all primary servers fail
- Dynamic backend discovery via server-template + DNS resolvers
- Built-in dashboard
- Rate limiting and connection throttling built-in

6.3. Web Architecture

6.3.1. Server-Side rendering

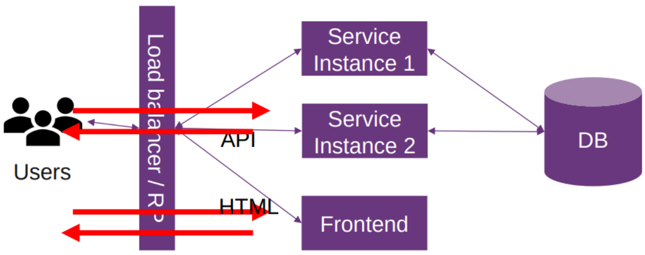
- Classic approach: SSR
- Server generates HTML/JS/CSS (dynamically)
  - User request: browser sends a request to the web server (server-side routing)
  - Server processing: server processes request by running server-side code (C#, Java, …)
    - May require data from database or other sources
    - Server-side code can use template engines or a framework to render the HTML
  - Response: Generate the appropriate HTML, CSS and JavaScript for the requested page
  - Browser rendering: browser receives response and renders page
- Advantage: SEO, immediate display
- Disadvantage: server rendering for every request (caching), UI logic on the server
- Static site generation (SSG): pre-render HTML/CSS/JS: only once, regenerate if content changes



6.4. Single Page Application SPA / CSR

- Interactions occur within a single web page
- App-like experience: client page dynamically updates as user interacts
- Relies on JavaScript to update UI, typically:
  - Initial response: server returns a single HTML file with references to CSS/JavaScript
  - Browser rendering: shows initial empty HTML file with a spinner, then executes JavaScript, then shows UI

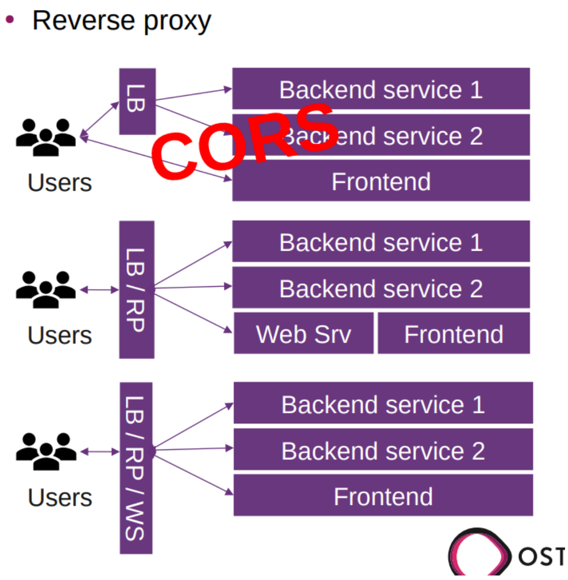
- User interactions: JavaScript manages the UI updates. Application does not require full page reloads. When you click a link in an SPA, instead of making a traditional HTTP request:
  - JavaScript intercepts the click events
  - Prevent default browser navigation
  - Update the URL using the History API
  - Render new content without requesting new HTML document, but may involve fetching data
- Fetching data: When the SPA needs to fetch or send data, communicates through APIs
- Use a framework: React, Angular, Vue
- Backend serves API requests only
- SEO only works if JavaScript is executed
  - Crawler gets JavaScript code, needs to execute, then it knows the content
- Good separation: UI in HTML/CSS/JS, backend in /api
- Client-side routing: SPAs for navigation
  - Server side routing -> default to index.html
- Typical setup
  - `/ -> index.html`
  - `/user -> user.html`



- Initial load: entire page
- Further requests: only updates partially

6.4.1. CORS

- CORS = Cross-Origin Resource Sharing
- For security reasons, browsers restrict cross-origin HTTP requests initiated from scripts
- Mechanism to instruct browsers that run a resource from origin A to run resources from origin B
- Solution: reverse proxy with webserver
  - Many solutions, caddy, vite/webpack dev server, nginx, mix -> the client only sees the same origin for the API and the frontend assets
- "Workaround": Access-Control-Origin: https://…
  - For dev: Access-Control-Origin: *
  - Pre-flight requests
  - Golang: w.Header().Set("Access-Control-Origin", "*")



6.4.2. SSR vs CSR

Performance Metrics:

- TTFB (Time to First Byte): Server response time
- FCP (First Contentful Paint): When content first appears
- TTI (Time to Interactive): When page becomes fully interactive
- Trade-off: SSR good at FCP/TTI, CSR requires full JS execution first

SSR vs CSR Initial Load:

- SSR: visible and interactive immediately (no JS needed)
- CSR: Must download, parse, execute JS before interactive
- Post-load: CSR good: no page reloads for navigation, feels like desktop appears

CSR Advantages:

- Lower server rendering load
- API only serves JSON

CSR Disadvantage:

- Bundle Size Problem: Large JS files, slow parse/execution, mobile may struggle
- Slow initial load: white screen until JS executes
- SEO problem: crawlers see empty html, need JS executing to read content

Why CSR? Clean architecture (frontend/backend separation)

6.4.3. CSR Improvements

- Code splitting, lazy loading, hybrid approach with hydration (when a page is first sent as HTML but still has to load JavaScript to make HTML interactive)
  - Hydration problems:
    - Duplicated work / complexity
    - Pre-rendering only: PrevelteKit
- Server Components (React)
  - Components render only on server, no JS sent to client

    - Reduces bundle size
- Server HTML fragments htmx: server replies with HTML fragments
- Island architecture
    - Static HTML with interactive islands
    - Only islands ship JS - minimal JS by default (Astro)
- Streaming SSR
    - Send HTML in chunks as ready
    - Browser renders earlier - improves perceived performance
- Edge rendering
    - Render closer to user geographically at CDN edge locations (Cloudflare Workers, Vercel Functions)

7. Week 6

7.1. Key takeaways

7.1.1. Explain what MCP is, why it exists, and how tool calling works (tools/list, tools/call, JSON-RPC)

MCP is an open protocol by Anthropic that connects AI agents to external tools, data sources, and services. It exists to avoid every AI app needing custom connectors for every tool. Tool calling uses JSON-RPC 2.0:

- tools/list: asks what tools are available
- tools/call: runs a selected tool with arguments

7.1.2. Describe the MCP message flow between user, client, LLM, and MCP server

The flow is: `user -> assistant (tool_call) -> tool (result) -> assistant (answer)`.

The LLM decides whether to call a tool, sends the tool call, receives the result, and then creates the final answer. Tool results must include the correct `tool_call_id`.

7.1.3. Discuss challenges of running MCP in the browser and how WebMCP addresses them

Browsers can block or limit external fetching, causing errors like `"could not fetch"`.

WebMCP solves this by using browser-side tools, for example a Firefox extension that injects a `client_web_search` tool and acts like an MCP server.

7.1.4. What authentication mechanisms exists for web applications?

- Passwords / single-factor authentication
- MFA / 2FA
- TOTP software tokens
- Basic Auth
- Digest Auth
- mTLS
- Session-based authentication
- JWT
- Access + refresh tokens
- OAuth

7.1.5. How can stateless authentication be achieved?

Stateless authentication can be achieved with JWTs.

After login, the server creates and signs a token. The client sends it on future requests with `Authorization: Bearer <token>`. Because every server instance can verify the token, the server does not need to remember a session.

7.2. Model Context Protocol (MCP)

- By anthropic, november 2024, open-source
  - Claude Opus / Sonnet
- Donated to linux foundation -> no single company controls it
- Connects AI agents to external tools, data sources and services
  - Universal Protocol
  - Implemented once, works everywhere
- Before MCP: every AI app needed a custom connector per tool (NxM problem)
  - Doesn’t scale, fragile, duplicated effort

- Born from a single devs frustration copy-pasting between claude desktop and his IDE
  - Not a corporate strategy, solving an annoying workflow problem

7.2.1. MCP API

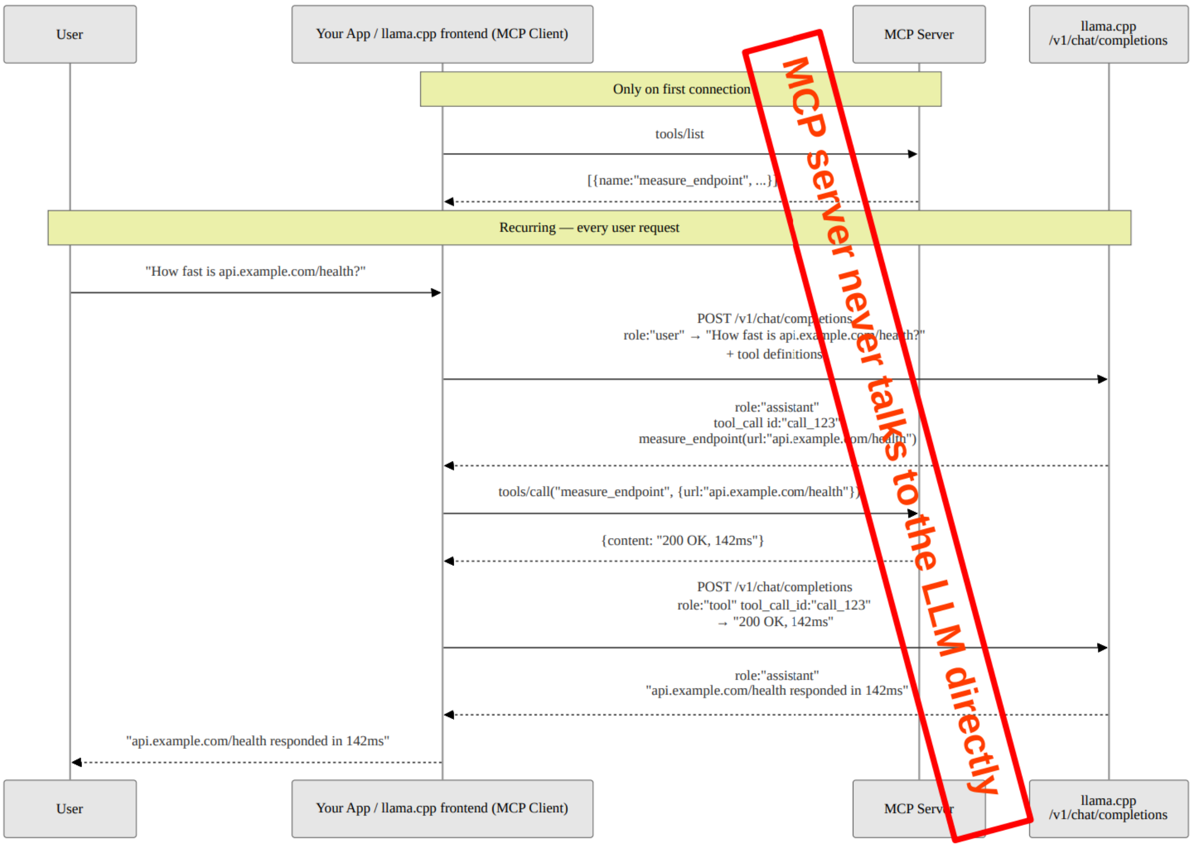
The MCP API uses JSON-RPC 2.0 for communication between an MCP client and an MCP server.

First, the client sends a `tools/list` request. This asks the server which tools are available. The server responds with an array of tools. Each tool contains a name, a description and an input schema that defines which arguments are required.

Afterwards, the client can use `tools/call` to execute one of these tools. The request contains the tool name and the required arguments. The MCP server then runs the tool and returns the result.

In short, `tools/list` is used to ask “what can you do?”, while `tools/call` is used to say “run this tool with these arguments”.

7.2.2. MCP Sequence Diagram



7.2.3. Simple MCP Server

A simple MCP server can expose tools that an LLM is allowed to call. In this example, the server provides one tool called `measure_endpoint`, which measures the response time of a given URL.

The available tools must be included in the LLM request. Each tool has a name, a description and a parameter schema. Based on the user request, the LLM decides whether a tool should be called, which tool to call and which arguments to use.

The usual role flow is:

```
user -> assistant (tool_call) -> tool (result) -> assistant (answer)
```

This means the user first asks a question, the assistant decides to call a tool, the tool returns the result and the assistant uses that result to generate the final answer.

The OpenAI Chat API is stateless, which means previous tool calls and their results are not remembered automatically. Therefore, after the tool has been executed, a second call to the LLM is required. This second request includes the original user message, the assistant message containing the tool calls and the tool result messages.

Each tool result must reference the correct `tool_call_id`. The LLM generates this ID when it requests a tool call and the application must echo it back in the matching tool response. If multiple tools are called, multiple `role: "tool"` messages must be returned, each with its own matching `tool_call_id`.

Using the wrong role or an incorrect `tool_call_id` can lead to errors or incorrect generated results, because the LLM can no longer reliably match tool outputs to the requested tool calls.

7.2.4. MCP in the Browser

- The client sends the available tools together with the user query.
- The LLM returns tool calls with names and arguments.
- The tool result is sent back to the LLM to generate the final answer.

Problem: A normal LLM running in the browser cannot reliably access external websites or fetch linked resources by itself. Browser restrictions, missing tool access or rate limits can cause errors such as “could not fetch”. Therefore, the model needs a way to use browser-side tools.

- A Firefox extension can intercept API requests and inject a `client_web_search` tool.
- The extension acts like the MCP server, so no separate MCP server is needed.
- This allows the LLM to search or fetch web content through the browser.
- In the future, WebMCP could allow websites to expose tools directly through the browser.

7.3. Authentication

- Single-factor authentication, e.g password
- Multi-factor authentication / 2FA, e.g password and software token, SMS unsecure
- Password rules - don’t:
  - Name of a pet, child, family member, or significant other
  - Anniversary dates and birthdays, birthplace
  - Name of a favorite holiday
  - The word “password”
- Don’t reuse passwords, use password managers
- Don’t enter passwords on unencrypted sites
- Use Argon2id for password storage
- Hashtype: WPA/WPA2
- Software token: TOTP (Time-based One-time Password)
  - Often used as 2nd factor (e.g Google Authenticator)
  - Based on keyed-hash message authentication code
  - hash(key + message)
  - K = shared secret
  - T = current unix time / 30s
  - TOTP(K, T) = Truncate(HMAC-SHA-256(K,T))
- Where should auth happen?
  - In service, e.g your HTTP server
  - In load balance, e.g traefik, jwt

7.3.1. Information security - key concepts / access control

- Confidentiality
  - Protects against eavesdroppers
- Integrity
  - Protection against data modification
- Availability
  - Data needs to be available when needed
- Non-repudiation
  - Neither the sender nor the receiver can deny that a communication has taken place
- Identification
  - E.g with a username “alice”, claiming to be Alice
- Authentication
  - Verifying a claim of identity, e.g alice shows passport, authentication types:
    - Something you know: things such as a PIN, a password
    - Something you have: a key, a swipe card
    - Something you are: biometrics: fingerprint
- Authorization
  - What resources an authenticated user is permitted to access

7.3.2. Basic Auth

Client sends a username and password with a HTTP request.

- Should be used with HTTPS
  - With basic auth the password isn’t encrypted
  - Password gets Base64 encoded
    - Base64: encoding format, not security. Can be decoded by anyone
  - Client will provide something like: `Authorization: Basic <base64>`
- Can be encoded in URL
- Server will reply with header
  - WWW-Authenticate: Basic realm=“restricted area”
  - The user will see the information “restricted area”

7.3.3. Digest Auth

- Hash + nonce, against replay attacks
- Server send
  - `WWW-Authenticate: Digest realm="testrealm@host.com"`
- Client sends in HTTP header
  - `Authorization: Digest username="Alice"realm="testrealm@host.com", nonce="dcd98b7102dd2f0e8b11d0f600bfb0c093", uri="/dir/index.html"`
- Advantages
  - PW not in clear text (MD5), can be “SHA-256”, “SHA-256-sess”, “SHA-512” and “SHA-512-sess”
    - sess: “session key” for “authentication session”
  - Nonce for replay protection for client and server
- Disadvantages
  - Browser L&F
  - Cannot use scrypt or bcrypt to store PWs

7.3.4. mTLS

Basic explanation: With mTLS both sides verify each other. e.g:
Client checks: Is this really the server?
Server checks: Is this client allowed?

- Create SSL CA certificates for server with openssl command
- Create CA / server cert
- Create client certificate
- Add caddy security in your local network (tls_client_auth)
  - If the certificate is missing or invalid inside caddy then the request is rejected

7.3.5. Lets Encrypt

- Free, automated, open certificate authority
- Provides domain control via challenges
- HTTP-Challenge verification
  - Server must be publicly reachable
  - Token replaced at /.well-known/acme-challenge/
  - Let's encrypt verifies token via HTTP request
- DNS-Challenge verification
  - Server does not need to be publicly reachable
  - Enables wildcard certificates (valid for all subdomains)
  - Token placed as TXT record in DNS
  - Let's encrypt verifies the DNS entry
- Certificates valid for 90 days (currently)
  - May 2026: opt-in 45-day certificates
  - Feb 2027: default 64-day certificates
  - Feb 2028: default 45-day certificates
  - Optional: 6-day short-lived certificates (since 2026)
- Integrated in modern web servers (e.g caddy)

7.3.6. Session-based authentication (stateful)

- The server remembers that a user is logged in.
- This can lead to problems with multiple service instances because the session is only stored in the memory of that instance
- E.g SpringBot (JSESSIONID)

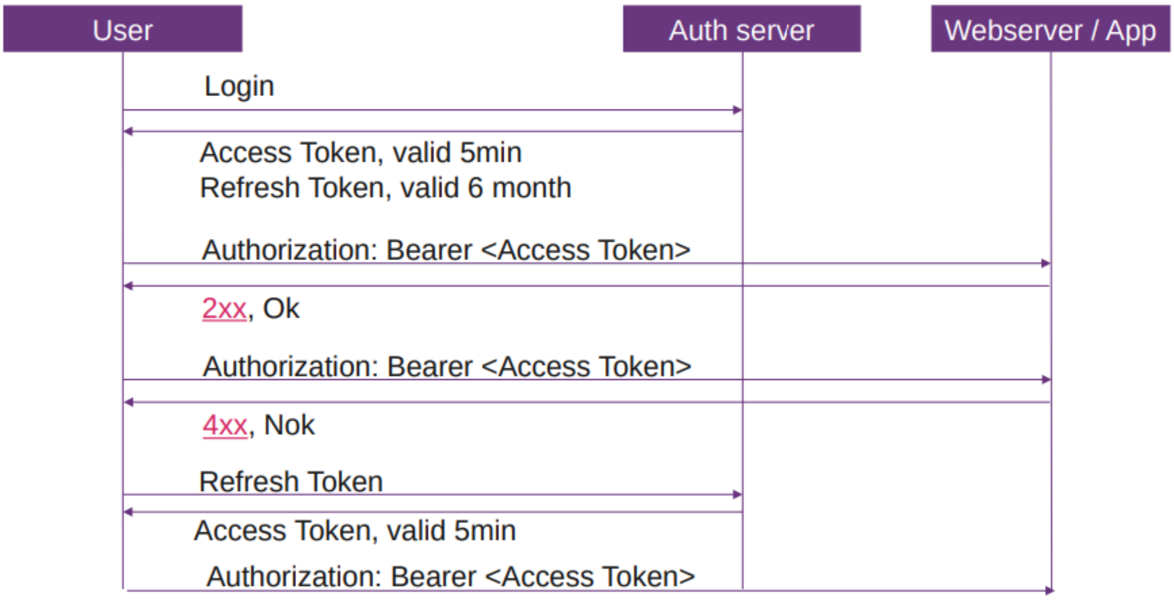
7.3.7. JSON Web Token (JWT) (stateless)

- JSON-based access tokens
  - Header: {“alg”:“HS256”}
  - Payload: {“sub”:“tom”, “role”:“admin”,“exp”:“1422779638”}
- All server instances know a secret token / public key
- When user logs in, server send back token
- Client sends: Authorization: Bearer
- Token: const_user_token = base64urlEncoding(header) + ‘.’ + base64urlEncoding(payload) + ‘.’ + base64urlEncoding(signature)
- Signature (simple): keyed-hash message hash(base64(header)+base64(payload) + secret token)
- Client can store token in `localStorage.setItem("token", accessToken);`

Flow:

1. User logs in with username/password
2. Server checks if login is correct
3. Server creates a JWT
4. Server signs the JWT with the secret key
5. Server sends the finished JWT back to the client
6. Client sends this JWT with future requests

7.3.8. Access Token / Refresh Token



Access Token:
- Short lifetime, e.g 10min
- If public key / secret is known, the content in the token can be trusted
- Can have userId, role etc -> no need to query DB for those Information

Refresh Token:
- Longer lifetime, e.g 6 months
- Used to get a new access token
- IAM / Auth server creates access tokens

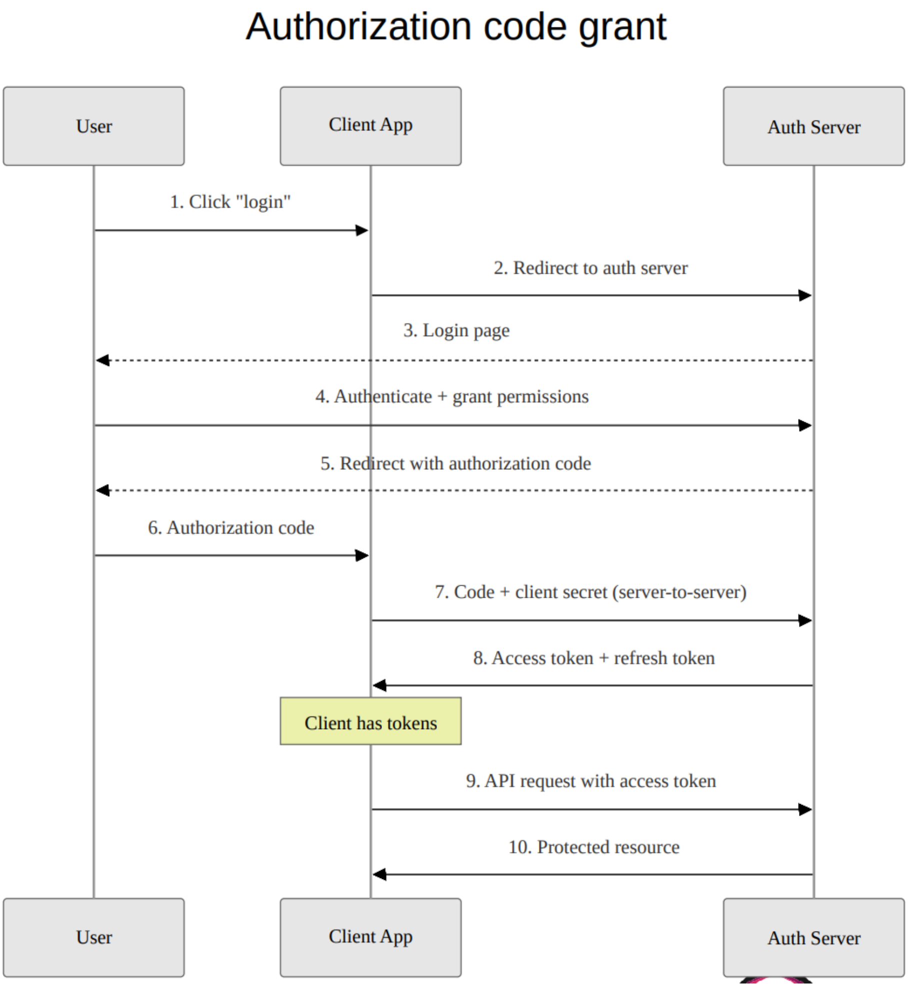
Only access token: If a user credential is revoked, how is every service informed?

Only refresh token: For every request to Service, Auth needs to be involved for access

Access + Refresh token: If a user credential is revoked, user has max. 10 min more to access service, but auth is only involved if the access token is expired

7.3.9. OAuth
- For authorization with 3rd party integration
  - Grant access without giving away passwords
- Authorization code grant
  - User redirected to authorization server
  - User authenticates and grants permission
  - Authorization code returned to app
- PKCE: variant for clients that cannot store a client secret securely (mobile, SPA)

- Token issued are typically JWTs (access + refresh)



8. Week 7 - Categorization

8.1. Key takeaways

8.1.1. What is a distributed system?

A distributed system is a group of computers working together so that, to the user, they appear like a single computer.

8.1.2. How can it be categorized?

There is no single universally applicable categorization. Distributed systems can be categorized by transparency, fault tolerance, scalability, consistency, data replication, data partitioning, and heterogeneity.

Examples are tightly vs. loosely coupled, homogeneous vs. heterogeneous, small-scale vs. large-scale, stateful vs. stateless, decentralized systems, client-server, peer-to-peer, hybrid, monolithic vs. microservices, and synchronous vs. asynchronous communication.

8.1.3. What are transparencies?

Transparencies are ways a distributed system hides its distributed nature from users.

Examples:
- Location transparency: users do not know the physical location
- Access transparency: users access resources in one uniform way
- Migration/relocation transparency: users do not notice resources moved
- Replication transparency: replicas appear as one resource
- Concurrent transparency: users do not notice other users
- Failure transparency: users do not notice recovery mechanisms
- Security transparency: users are minimally aware of security mechanisms

8.2. Distributed System Definition

A distributed system in its simplest definition is a group of computers working together as to appear as a single computer to the user

8.3. Distributed Systems Categorization

Classification: degree of transparency, fault tolerance, scalability, consistency (strong, eventual, and causal), data replication, data partitioning, heterogeneity.

There is no single universally applicable categorization of distributed systems

Classifications:
- Tightly coupled: Processing elements or nodes have access to a common memory
- Loosely coupled: Do not

Homogeneous System: All processors within the system have the same type
Heterogeneous System: Contains processors of different types

Small-scale system: WebApp + Database
Large-scale system: 2+ machines

Stateful: Maintains state between requests (e.g databases, session-based applications)
Stateless: System treats each request independently (e.g REST APIs behind a load balancer)

Decentralized: Distributed in a technical sense but not owned by one actor

More classifications:

- Based on architecture (Client-server, peer-to-peer, hybrid, software architecture)
- Based on deployment architecture (monolithic vs. microservices)
- Based on communication model (synchronous or asynchronous)

8.3.1. Controlled distributed systems vs. fully decentralized systems

- 1 responsible organization
- Low churn
- Examples:
- Amazon DynamoDB
- Client/server
- “Secure environment”
- High availability
- Can be homogeneous / heterogeneous
- Mechanisms that work well:
- Master nodes, central coordinator
- Consistent hashing (DynamoDB, Cassandra)
- Network is under control or client/server → no NAT issues
- Consistency
- Leader election (Zookeeper/Zab, Paxos, Raft)
- Replication principles
- More replicas: higher availability, higher

reliability, higher performance, better scalability, but: requires maintaining consistency in replicas

- Transparency principles apply

- N responsible organizations
- High churn
- Examples:
- BitTorrent
- Blockchain
- “Hostile environment”
- Unpredictable availability
- Is heterogeneous
- Mechanisms that work well:
- Consistent hashing (DHTs)
- Flooding/broadcasting - Bitcoin
- NAT and direct connectivity huge problem
- Consistency
- Weak consistency: DHTs
- Nakamoto consensus (aka proof of work)
- Proof of stake – Leader election, PBFT protocols, Is Bitcoin eventually consistent? — Some argue no, some argue it has even stronger guarantees
- Replication principles apply to fully decentralized systems as well
- Transparency principles apply

8.4. CAP theorem

States that distributed data store cannot simultaneously be consistent, available and partition tolerant. Choose between consistency or availability.

- Consistency: Every node has the same consistent state
- Availability: Every non-failing node always returns a response
- Partition Tolerant: The system is designed to deal with broken communication between the nodes.

Example: Cassandra (NoSQL) is designed to be AP but can be CP.

8.5. Transparency in distributed systems

Distributed systems should hide its distributed nature:

- **Location transparency:** users should be unaware of the physical Location
- **Access transparency:** users should access resources in a single uniform way
- **Migration, relocation transparency:** users should not be aware that resources have moved
- **Replication transparency:** users should not be aware about replicas, it should appear as a single resource
- **Concurrent transparency:** user should not be aware of other users
- **Failure transparency:** user should not be aware of recovery mechanisms
- **Security transparency:** users should be minimally aware of security mechanisms

8.6. Fallacies of Distributed Computing

"Fallacies of Distributed Computing" means: Things developers often assume about networks, but should not assume.

1. Network is reliable
   - Even submarine cables can fail
2. Latency is zero
   - Ping to AUS is 300ms
3. Bandwidth is infinite
   - Send a bike courier with an 8TB disk, that arrives 10h later, or send the data with a 1Gibt/s link? 8 **1000** 8 / (10 **60** 60) = 1.7Gbit/s
4. The network is secure
   - Assume someone is listening
5. Topology doesn't change
   - Ping to AUS request can take different route than reply
6. There is one administrator
   - Sometimes your route goes from one company to another rival company
7. Transport cost is zero
   - Someone builds and maintains the network
8. The network is homogeneous
   - WiFi, cable, desktop, server, mobile

9. Week 8 - Communication Protocols

9.1. Key takeaways

9.1.1. How do network layers work?

Network layers provide services to the layer above them. Protocols allow an entity at one layer to interact with the same layer on another host.

Each PDU contains a protocol header and a payload, called the SDU.

9.1.2. What are the TCP mechanisms?

TCP is reliable and ordered. It uses retransmission, sequence numbers, ACKs, checksums, flow control, and congestion control.

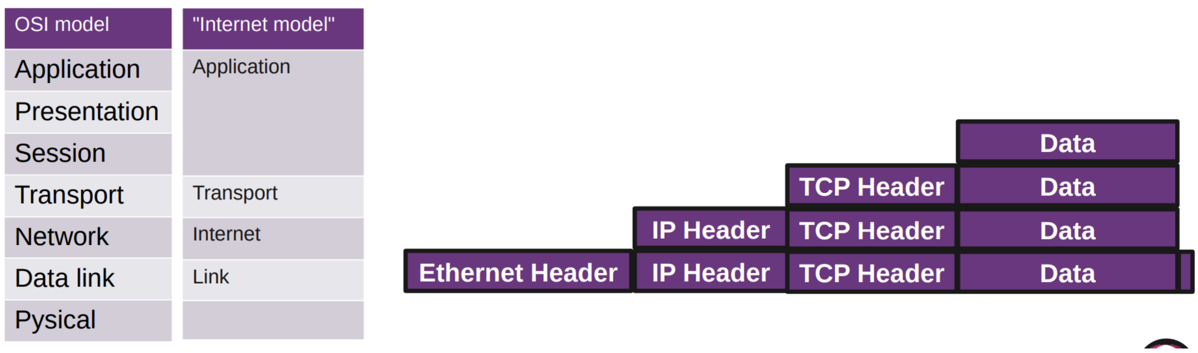
Connections are established with SYN, SYN-ACK, ACK and terminated with FIN and ACK messages. Lost data can be detected with retransmission timeouts, duplicate ACKs, and selective ACKs.

9.1.3. What are the problems of TCP, and how other protocols (HTTP/3) can improve that

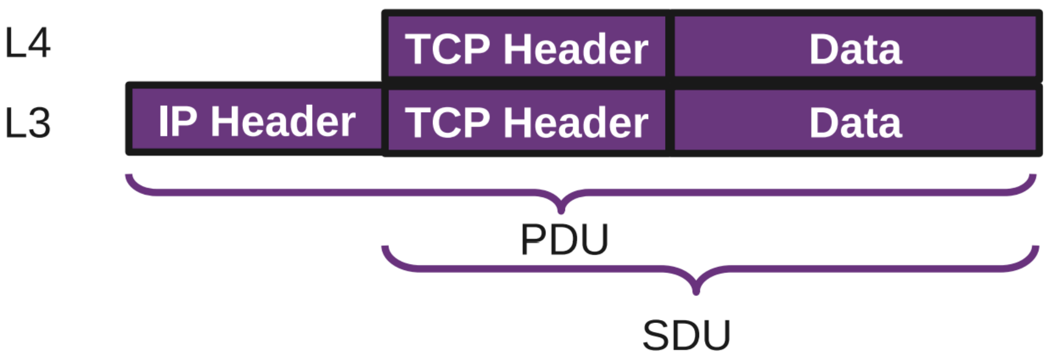
TCP handshakes are not very flexible and can require multiple round trips, especially with TLS and DNS. TCP also requires packets to arrive in order, so one missing packet can delay everything.

HTTP/3 uses QUIC, which has a 1 RTT connection and security handshake, 0 RTT for known connections, built-in security, and multiplexing where other streams can continue even if one stream has packet loss.

9.2. Networking Layers



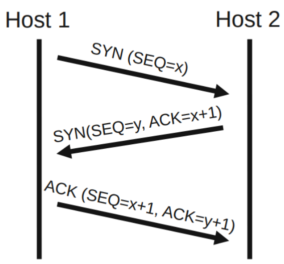
- Protocols enable an entity/instance to interact with an entity/instance at the same layer in another host
- Service definitions: provide functionality to an (N)-layer by an (N-1) layer
- Each PDU (protocol data unit) contains a protocol header and payload, the service data unit (SDU)

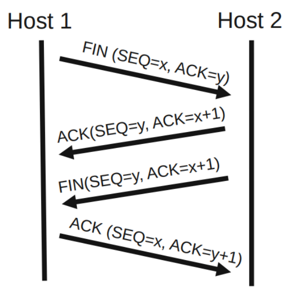


9.3. Layer 4 - Transport

9.3.1. TCP

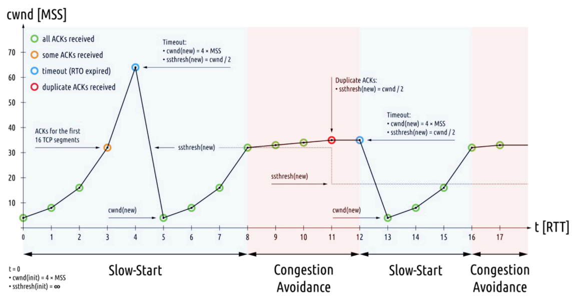
- Reliable (retransmission)
- Ordered
- Window - capacity of receiver
- Checksum - 16bit (crc16)
- TCP overhead: 20bytes (MTU 1500 bytes)
  - IP overhead: 20 bytes
  - Ethernet frame: 18 bytes (crc32)
- TCP tries to correct errors





- Connection establishment
  - SYN, SYN-ACK, ACK (three way)
  - Initiates TCP sessions: initial sequence number is random
- Connection termination
  - FIN, ACK + FIN, ACK (three/four way)
  - 3-way handshake, when host 1 sends a FIN and host 2 replies with a FIN & ACK
- Sequences and ACKs
  - Identification each byte of data
  - Order of the bytes -> reconstruction
  - Detecting lost data: RTO, DupACK
- Retransmission timeout
  - If no ACK is received after timeout (e.g 2xRTT), resend
- Duplicate cumulative acknowledgements, selective ACK
  - ACKs for last consecutive packets
  - 3 times same ACK -> retransmit missing packets (fast retransmit)

- Flow Control
  - Sender is not overwhelming a receiver
  - Back pressure
  - Sliding window:
    - Receiver specifies the amount of additionally received data in bytes that can be buffered
    - Sender up to that amount of data before the ACK
- Congestion Control
  - slow-start
  - congestion avoidance



9.3.2. TCP/IP from an Application Developer View

Problem: TCP handshake is not flexible

- You need a handshake (1RT)
  - If you want to make sure the other side accepts packets (and not drop it), ensure both sides are ready to transmit and receive data
  - If you want to exchange public / private keys
- TCP + Security = at least 2 RT

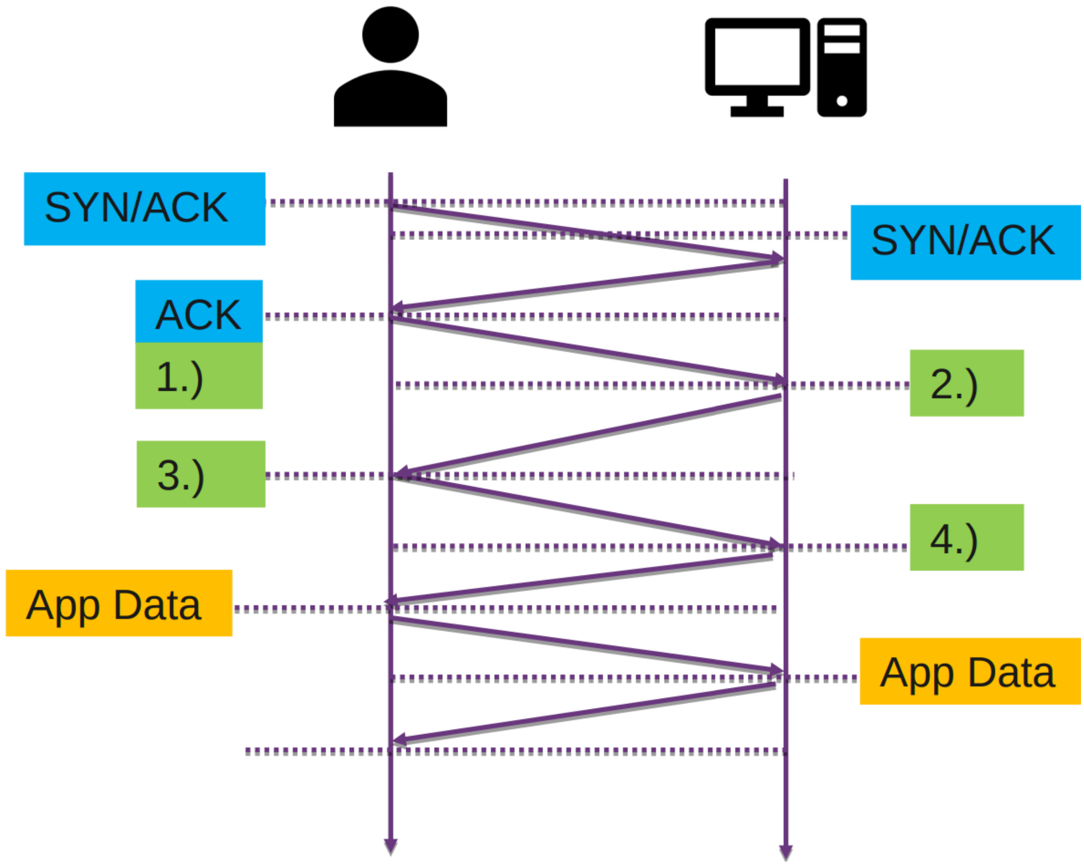
- DNS query may be required too: 3 RT
- Old security protocols add RT: 4 RT
- Worst case: 1.4s before data can be sent

9.3.3. TCP + TLS

Security: Transport Layer Security (TLS)

1. "client hello" lists cryptographic information, TLS version, ciphers/keys
2. "server hello" chosen cipher, the session ID, random bytes, digital certificate (checked by client), optional: "client certificate request"
3. Key exchange using random bytes, now server and client can calc secret Key
4. "finished" message, encrypted with the secret key

Total: 3 RTT to send first byte, 4 RTT to receive first byte



TLS 1.3: 1 RTT instead of 2

- Client Hello, Key Share
- Server Hello, key share, verify certificate, finished

9.3.4. QUIC / HTTP3

- QUIC: 1RTT connection + security handshake
- For known connections 0 RTT
- Built-in security
- HTTP/2 introduced multiplexing
- If one stream has packet loss, other streams can continue normally

The problem with HTTP/2: TCP requires packets to arrive in order, one missing packet can delay everything

Downsides of HTTP/3:
- NAT: TCP (SYN, ACK, FIN) knows when connection ends, with UDP you get many timeouts (creates many entries)
- HTTP header compression: Can reference previous headers which can create dependency
- Cannot benefit from TCP optimizations

9.3.5. UDP
- UDP is used for DNS, streaming audio and video
- Simple connectionless communication model
- No guarantee
  - Delivery
  - Ordering
  - Duplicate protection

9.3.6. Layer 4 - SCTP
- Message-based
- Allows data to be divided into multiple streams
- Syn cookies - SCTP uses a four-way handshake with a signed cookie
- Multi-homing multiple IP addresses of endpoints
- Not widely used
  - Problems with home routers
  - Used by WebRTC but tunneled over UDP

9.3.7. DDoS Amplification Attacks
DDoS amplification abuses UDP services that respond with more data than they receive. The attacker spoofs the victim’s address, so the server sends the large response to the victim. Tools like hping3 can create such spoofed UDP packets, while normal programming languages usually do not support this easily.

9.3.8. Comparison

TCP	UDP	SCTP	QUIC
Transport layer	Transport layer	Transport layer	Transport layer
Connection oriented	Connection less	Connection oriented	Connection oriented
Reliable transfer	Unreliable transfer	Reliable transfer	Reliable transfer
Streams	Messages	Messages	Multistream
Guaranteed order	Unordered	User can choose	Guaranteed order
Widely used - HTTP/1, HTTP/2	Widely used - DNS, HTTP/3	WebRTC	HTTP/3
Flow and congestion control	No flow, congestion	Flow and congestion control	Flow and congestion control
Heavyweight	Lightweight	Heavyweight	Heavyweight
Error checking and recovery	Error checking, no recovery	Error checking and recovery	Integrity check

Table 1: Comparison of transport protocols

9.4. Alternatives to QUIC

There are several alternative transport protocols or libraries, but most have drawbacks:

- KCP: A transport protocol without built-in encryption.
  - GFCP: A variant of KCP.
- UDT: No longer maintained.
- utp4j: Java implementation of Micro Transport Protocol, later handed over to Tribler at Delft University of Technology.
- Other related projects include:
  - 0-RTT protocol in Go
  - ATP, a peer-to-peer protocol
  - P2P library in Go

9.5. QOI - The quite ok image format

- Simple version of png
- Enocoder / decoder in 300 loc
- PNG generates smaller images, QOI is much faster
- Compression not much worse but simpler

10. Week 9 - Application Protocols

10.1. Key takeaways

10.1.1. Overview over important protocols on layer 7

10.1.2. Custom protocols, ASN.1, RPC, HTTP, JSON, WebSockets, Server Sent Events

10.1.3. Bencoding, WebRTC, DNS, Let's Encrypt, Mail Protocols

10.2. Protocols

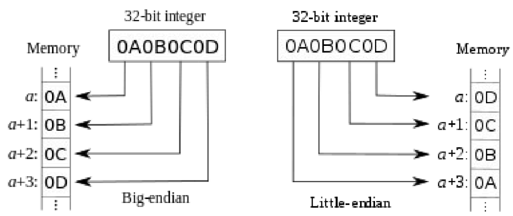
Custom Protocols:
+ less space
+ faster encoding/decoding (custom)
+ fewer unecessary fields
+ better performance

- development
- testing/debugging
- compatibility
- documentation

Protocol generators:
- Thrift: RPC framework from facebook, IDL and binary protocol
- Avro: data serialization system, remote procedure call and data serialization framework - Hadoop (Big-data framework)
- Protocol Buffers: data serialization from Google, IDL -> goals: smaller and faster than XML
- ASN.1: serializ. / deserializ, used for certs

IDL = Interface Description Language -> describe the protocol once, the tool generates code for different programming languages

Little-endian / big endian:
- Sequential order where bytes are converted into numbers
- Networking (TCP headers): Big-endian
- Most CPUs: Little-endian



10.3. RPC Examples

Remote Procedure Call: lets programs execute code on remote machines.

Examples:
- gRPC: Uses HTTP/2 for transport, uses Protocol Buffers
- Features: Authentication, bidirectional streaming and flow control, blocking or nonblocking bindings, and cancellation and timeouts

10.3.1. JSON/REST/HTTP

Human-readable text to transmit data. Can sometimes be RPC:
- REST ideally stateless, RPC can be stateful
- REST responses contain all metadata, RPC often needs interface definitions

- Parsing overhead: JSON slower than binary protocol
- Often used for web apps

So JSON is just the data format, HTTP is the transport, and REST/RPC describe the API style.

10.4. Application Protocol: HTTP

- Foundation of web data communication
- Request/response model, resources identified by URL
- URL structure: scheme, user info, host, port, path, query, fragment
- Stateless (server keeps no state)
- Browser sends additional headers (User-Agent, Accept, Accept-Encoding)

Scheme	User info	Host	Port	Path	Query	Fragment
http://	tbocek:password@	dsl.i.ost.ch	:443	/lect/fs21	?id=1234&lang=de	#topj

Response: header + status code + content body

Header:

Response Headers (227 B)

```
HTTP/2 200 OK
server: cloudflare-nginx
content-type: text/html; charset=UTF-8
date: Mon, 02 Mar 2020 14:29:39 GMT
x-page-speed: 1.13.35.2-0
cache-control: max-age=0, no-cache
content-encoding: gzip
X-Firefox-Spdy: h2
```

Status Codes:

- 1xx (informational),
- 2xx (success – 200 OK),
- 3xx (redirection),
- 4xx (client error – 404 Not Found, 403 Forbidden),
- 5xx (server error)

Body/Content: <!DOCTYPE html> …

Request Methods: GET, HEAD, POST, PUT, DELETE, TRACE, OPTIONS, CONNECT, PATCH

10.5. WebSockets

- Full-duplex (two way) communication over TCP (REST/JSON is in one direction)
- HTTP handshake, then upgrade to communication channel (WebSocket)
- With SSL/TLS -> `wss://`
- Example usages: WhatsApp, Google Docs, YouTube
- Many languages support it, e.g Golang
- How can the server notify the browser
  - Polling:
    - Short: Request e.g every 0.5s
    - Long: request until timeout or reply
  - Server Sent Events (alternative) SSE
    - One way communication from server to browser
    - Server receives a regular HTTP request, keeps connection open (Accept or Content-Type: text/event-stream)
    - Native browser support via EventSource API
    - Automatic reconnection if connection is lost
    - Max number of concurrent connections per domain

10.6. WebRTC

- Browser to browser communication (P2P)
- Real time communication (RTC) via API
- Goal: eliminate plugins or native apps
- Supported by: Google, MS, Mozilla, Opera, Apple
- Replaced need for Flash, Java plugins -> allows e.g Chrome <-> Communication
- Used in WhatsApp, Facebook Messenger, MS Teams etc.

10.6.1. Concerns

- HTML browsers get bloated
  - Several GB RAM to open couple of tabs (Hint: adblocker)
- WebRTC API could be simplified
- Security concerns: May reveal IP information despite using VPN/tor
- WebRTC forbids unencrypted communication
  - DTLS (data), SRTP (media)
  - Complexity - SCTP over DTLS over UDP

11. Week 12

11.1. Key takeaways

11.1.1. Understand how Bitcoin works as a peer-to-peer system

Bitcoin has no central bank, transactions are created by peers and broadcast to the whole network.

11.1.2. Get familiar with UTXO, mining, and proof-of-work

Coins are spent as whole UTXO pieces, while miners create valid blocks by solving proof-of-work puzzles.

11.1.3. Be aware of risks like the 51% attack

An attacker with more than 50% computing power could influence transaction ordering and exclude transactions.

11.1.4. Reflect on advantages and disadvantages of Bitcoin

Bitcoin is decentralized and can have low fees and strong anonymity, but has high power use, volatility and scalability limits.

11.2. Introduction

- Bitcoin is an experimental digital currency.
- It is fully peer-2-peer, with no central entity like a central bank.
- It was first issued on Jan 3, 2009.
- The smallest unit is 0.00000001 BTC, called 1 satoshi.
- The initiator is unknown.
- Bitcoin can be exchanged for real currencies.
- There is a maximum of ~21 million BTC.
- Every transaction is broadcast to all peers, so they all know every transaction (`~736 GByte`).
- Transactions are validated by proof-of-work, a partial hash collision that is difficult to fake.
- Bitcoin does not rely on trust, but on strong cryptography.
- Bitcoin has weak anonymity.
  - Anonymity can be weakened through clustering. For example, if a transaction has multiple input addresses, those addresses are assumed to belong to the same wallet.
- BIP: Bitcoin Improvement Proposals

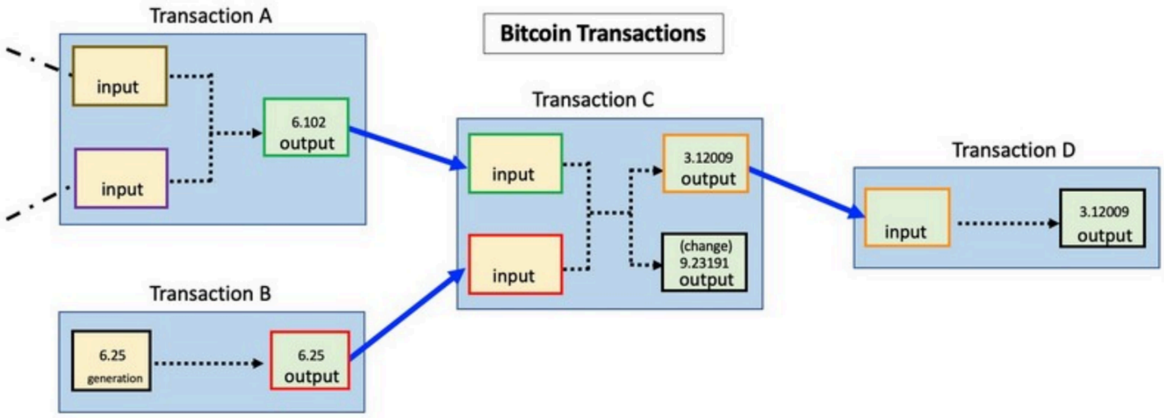
11.3. Mechanism

A wallet has public-private keys (wallet.dat)

- Public key, ECDSA 256 bit → Bitcoin address (can receive bitcoins)
- Simple address base58(RIPEM160(Sha256(ecdsa public key)))
  - E.g. 1GCeaKuhDYnNLNR6LGmBtKhPqEJD4KeEtF
- Private key used for signing transactions

11.3.1. Transaction

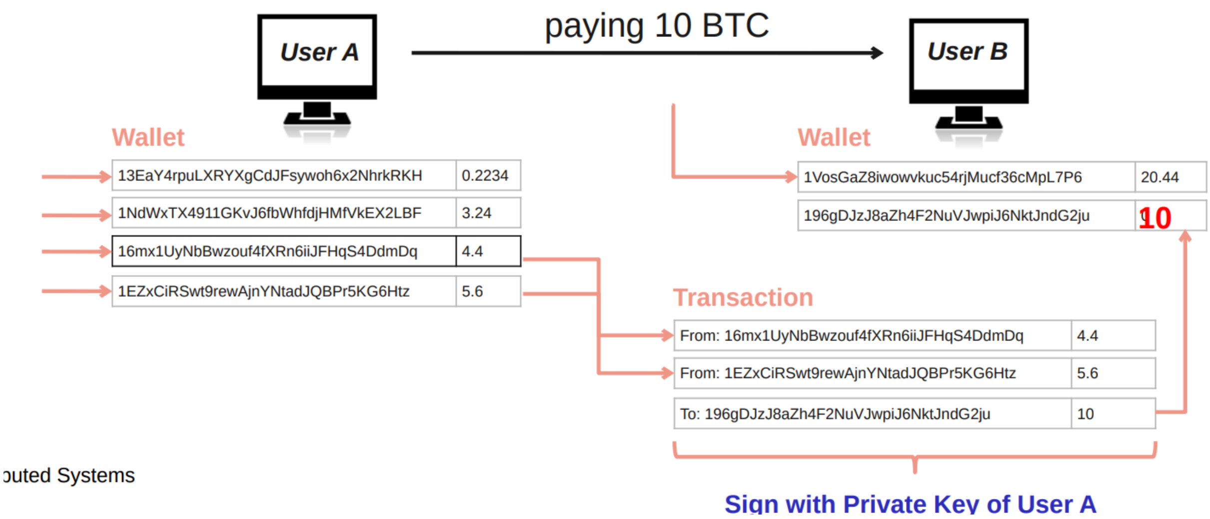
- Peer A wants to send BTC to peer B → creates transaction message
- Transaction contains input / output
  - where the BTC came from and where it goes
- Peer A broadcasts the transaction to all the peers in the network
- Transaction stored in blocks → block is created / verified 10min



11.4. Key Bitcoin Operation

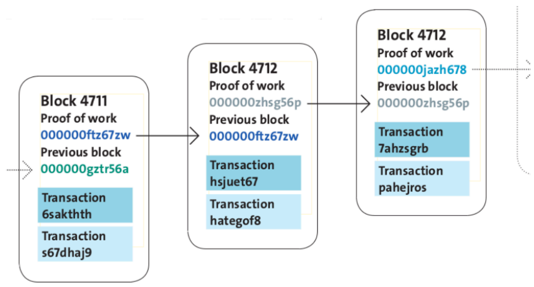
Private key authorizes the transation:

- Stolen keys can be used by thieves
- Key lost = coins lost
- In UTXO (unspent transaction output) systems, coins are stored as unspent pieces from earlier transactions
- When one piece is used, the whole piece is spent, the rest comes back as new change



11.5. Blockchain

- Transactions are collected in blocks
  - New block created approximately every 10 min
- Blocks contain solved crypto puzzles
  - In the form of partial hash collisions (SHA256)
  - Miners search for a block hash with a required pattern, e.g. many leading zeroes
  - More required zeroes = higher difficulty
- A block has a pointer to previous block →
  - This creates the blockchain



    - Changing an old block changes its hash, so all following blocks would need to be recalculated
- Creation of blocks is called mining (reward)
    - Mining / creating blocks → Miner get currently 3.125 BTC per creation
        - adjustable difficulty 6 blocks / h
        - Sometime in 2028 reward will be 1.5625

11.6. Mining mechanism

There are a couple of big miners (e.g the 1 million bitcoins of satoshi whose whereabouts are unknown).

- Mining = creating valid blocks
- Blocks are linked to previous blocks
    - Longest block survive (most difficult)
- Different level of confirmations
    - 3-6 block conf. is considered secure
- Dangerous if someone has more than 50% computing power
    - Can exclude and modify the ordering of transactions

Mining evolution:
1. CPU
2. GPU
3. FPGA (field programmable gate array)
4. ASIC Farms
    1. Application-Specific Integrated Circuit
    2. Whole datacenter

ASIC scenario:
- Old miner with 70 GHash/s
- Generates `~0.004 CHF` per day in 2026
- Uses 700W
    - 16.8kWh
    - Cost per day 3.69 CHF -> (Hochtarif, 0.2CHF per kWh)
    - Cost per day 2.18 CHF -> (Niedertarif, 0.15CHF per kWh)

11.7. Discussion

Advantages:
- Low tx fees
- Scalable (hardware/storage gets faster)
- Anonymity (preserving privacy)
- No major crashes
- Decentralized
- Other blockchain use cases

Disadvantages:
- High power consumption
- Not scalable (less transactions per sec than visa)
- Anonymity (illegal activities)
- Volatile exchange rates
- Central elements (devs)